

UNIVERSITÉ JOSEPH KI-ZERBO

Institut de Formation Ouverte à Distance (IFOAD)

L I C E N C E 3

P R O J E T — A P P L I C A T I O N M O B I L E

DOC.AGENDA

Application mobile de gestion de rendez-vous
(Centre de Santé)

Réalisé par

DIALLO Boukary

Professeur

M. Dieudonné GANSBEOGO

2025 — 2026

1. INTRODUCTION

CONTEXTE

Dans le cadre du module de développement mobile en Licence 3 à l'IFOAD, il nous a été demandé de concevoir une application mobile permettant de gérer les rendez-vous d'un centre de santé. L'objectif est de moderniser la prise de rendez-vous afin de réduire les files d'attente, améliorer l'organisation interne et permettre aux patients de prendre rendez-vous à distance.

L'application, nommée **DocAgenda**, permet aux patients de consulter les médecins disponibles, de prendre rendez-vous en ligne et de suivre l'état de leurs rendez-vous. Le personnel médical (administrateur) peut gérer les médecins, leurs disponibilités et valider ou refuser les rendez-vous. Le tout est connecté à une API REST Node.js/Express avec une base de données MySQL.

OBJECTIFS

- Maîtriser React Native pour le développement mobile
- Concevoir et consommer une API REST depuis une application mobile
- Implémenter une authentification JWT avec gestion des rôles (Patient / Admin)
- Réaliser un CRUD complet sur plusieurs ressources (médecins, rendez-vous, patients)
- Mettre en place un système de notifications in-app
- Adopter une architecture MVC côté backend

TECHNOLOGIES UTILISÉES

React Native 0.83

Node.js

Express

MySQL

React Navigation

JWT

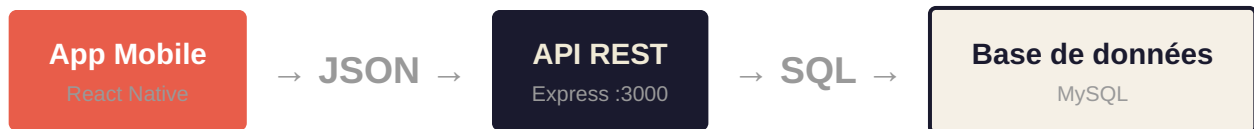
bcryptjs

AsyncStorage

2. ARCHITECTURE GÉNÉRALE

VUE D'ENSEMBLE

L'application suit une architecture client-serveur classique en trois couches :



L'application mobile communique avec le backend via des requêtes HTTP (`fetch`) au format JSON. Le backend Express, structuré en architecture MVC (Models / Controllers / Routes), expose les endpoints REST et exécute les requêtes SQL paramétrées sur la base MySQL via le driver `mysql2`.

STRUCTURE DES FICHIERS

```
DocAgenda/
├── backend/
│   ├── server.js           Point d'entrée Express
│   ├── db.js              Pool de connexions MySQL
│   ├── setup-db.js        Création tables + données de test
│   ├── middleware/
│   │   └── auth.js        Vérification JWT + contrôle de rôle
│   ├── models/
│   │   ├── userModel.js   Accès données utilisateurs
│   │   ├── medecinModel.js Accès données médecins
│   │   ├── rendezVousModel.js Accès données rendez-vous
│   │   └── notificationModel.js Accès données notifications
│   ├── controllers/
│   │   ├── authController.js Inscription, connexion, profil
│   │   ├── medecinController.js CRUD médecins + créneaux
│   │   ├── rendezVousController.js Gestion RDV + notifications auto
│   │   ├── patientController.js Liste et suppression patients
│   │   └── notificationController.js
│   └── routes/
│       ├── auth.js         medecins.js         rendezVous.js
│       ├── patients.js     disponibilites.js
│       └── notifications.js
├── mobile/
│   ├── App.tsx             Navigation principale
│   ├── api.js              Appels fetch vers l'API
│   ├── contexts/           AuthContext + NotificationContext
│   ├── hooks/              useAuth, useMedecins, useRendezVous, useNotifications
│   └── components/         Bouton, ChampTexte, CarteMedecin, CarteRendezVous...
```

	└─ screens/	16 écrans (Patient + Admin)
└─	compose.yml	Configuration Docker MySQL

3. BASE DE DONNÉES

La base de données `docagenda` est composée de cinq tables.

TABLE `USERS`

Stocke les patients et les administrateurs. Le champ `role` distingue les deux types d'utilisateurs.

COLONNE	TYPE	DESCRIPTION
<code>id</code>	INT, PK, AUTO_INCREMENT	Identifiant unique
<code>nom</code>	VARCHAR(100), NOT NULL	Nom de famille
<code>prenom</code>	VARCHAR(100), NOT NULL	Prénom
<code>email</code>	VARCHAR(150), UNIQUE	Email (identifiant de connexion)
<code>telephone</code>	VARCHAR(20)	Numéro de téléphone
<code>password</code>	VARCHAR(255), NOT NULL	Mot de passe hashé (bcrypt)
<code>role</code>	ENUM('patient', 'admin')	Rôle de l'utilisateur
<code>created_at</code>	DATETIME	Date d'inscription

TABLE `MEDECINS`

COLONNE	TYPE	DESCRIPTION
<code>id</code>	INT, PK, AUTO_INCREMENT	Identifiant unique
<code>nom</code>	VARCHAR(100), NOT NULL	Nom du médecin
<code>specialite</code>	VARCHAR(100), NOT NULL	Spécialité médicale
<code>created_at</code>	DATETIME	Date d'ajout

TABLE `DISPONIBILITES`

Plages horaires pendant lesquelles un médecin est disponible, par jour de la semaine.

COLONNE	TYPE	DESCRIPTION
id	INT, PK, AUTO_INCREMENT	Identifiant unique
medecin_id	INT, FK → medecins(id)	Médecin concerné
jour	ENUM('lundi'...'dimanche')	Jour de la semaine
heure_debut	TIME	Début de la plage
heure_fin	TIME	Fin de la plage

TABLE RENDEZ_VOUS

COLONNE	TYPE	DESCRIPTION
id	INT, PK, AUTO_INCREMENT	Identifiant unique
patient_id	INT, FK → users(id)	Patient qui a pris le RDV
medecin_id	INT, FK → medecins(id)	Médecin choisi
date_rdv	DATE	Date du rendez-vous
heure_rdv	TIME	Heure du rendez-vous
statut	ENUM('en_attente', 'confirme', 'annule')	État du RDV
motif	VARCHAR(255)	Motif de la consultation
created_at	DATETIME	Date de création

TABLE NOTIFICATIONS

COLONNE	TYPE	DESCRIPTION
id	INT, PK, AUTO_INCREMENT	Identifiant unique
user_id	INT, FK → users(id)	Destinataire
titre	VARCHAR(100)	Titre de la notification
message	VARCHAR(255)	Contenu du message
type	ENUM('creation', 'confirmation', 'annulation')	Type d'événement
rdv_id	INT, FK → rendez_vous(id)	RDV concerné

lue	BOOLEAN	Lue ou non
created_at	DATETIME	Date de la notification

4. API REST

Le backend expose six groupes de routes. Toutes les données sont échangées au format JSON.

ENDPOINTS AUTHENTIFICATION

MÉTHODE	ROUTE	DESCRIPTION	ACCÈS
POST	/api/auth/register	Inscription patient	Public
POST	/api/auth/login	Connexion (retourne un token JWT)	Public
GET	/api/auth/me	Profil de l'utilisateur connecté	Auth

ENDPOINTS MÉDECINS

MÉTHODE	ROUTE	DESCRIPTION	ACCÈS
GET	/api/medecins	Liste de tous les médecins	Auth
GET	/api/medecins/:id	Détail d'un médecin	Auth
POST	/api/medecins	Ajouter un médecin	Admin
PUT	/api/medecins/:id	Modifier un médecin	Admin
DELETE	/api/medecins/:id	Supprimer un médecin	Admin
GET	/api/medecins/:id/disponibilites	Disponibilités d'un médecin	Auth
POST	/api/medecins/:id/disponibilites	Ajouter une disponibilité	Admin
GET	/api/medecins/:id/creneaux?date=	Créneaux libres à une date	Auth

ENDPOINTS RENDEZ-VOUS

MÉTHODE	ROUTE	DESCRIPTION	ACCÈS
POST	/api/rendez-vous	Créer un rendez-vous	Patient
GET	/api/rendez-vous/mes-rdv	Mes rendez-vous	Patient

GET	/api/rendez-vous	Tous les rendez-vous	Admin
GET	/api/rendez-vous/:id	Détail d'un RDV	Auth
PUT	/api/rendez-vous/:id/statut	Changer le statut	Admin
DELETE	/api/rendez-vous/:id	Annuler un RDV	Auth

ENDPOINTS PATIENTS

MÉTHODE	ROUTE	DESCRIPTION	ACCÈS
GET	/api/patients	Liste des patients	Admin
GET	/api/patients/:id	Détail d'un patient	Admin
DELETE	/api/patients/:id	Supprimer un patient	Admin

ENDPOINTS NOTIFICATIONS

MÉTHODE	ROUTE	DESCRIPTION	ACCÈS
GET	/api/notifications	Mes notifications	Auth
GET	/api/notifications/count	Nombre de non lues	Auth
PUT	/api/notifications/lire-tout	Tout marquer comme lu	Auth
PUT	/api/notifications/:id/lire	Marquer une comme lue	Auth

SÉCURITÉ

- Mots de passe hashés avec `bcryptjs` (10 rounds) avant stockage
- Authentification par tokens JWT avec une durée de validité de 7 jours
- Middleware `auth` vérifiant le token Bearer sur chaque route protégée
- Middleware `adminOnly` restreignant certaines routes aux administrateurs
- Requêtes SQL paramétrées pour prévenir les injections SQL
- Suppression d'un patient bloquée s'il a des rendez-vous actifs

5. APPLICATION MOBILE

NAVIGATION

La navigation est gérée par `React Navigation`. L'application affiche un flux différent selon l'état de l'utilisateur :

- **Splash Screen** : écran d'accueil affiché pendant 2 secondes au lancement
- **Non connecté** : écrans Connexion et Inscription (pile d'authentification)
- **Connecté en tant que Patient** : barre d'onglets avec 5 sections :
 - **Accueil** → Tableau de bord avec statistiques des RDV
 - **Médecins** → Liste des médecins → Prise de rendez-vous
 - **Mes RDV** → Liste des rendez-vous → Détail du RDV
 - **Notifications** → Liste des notifications avec badge
 - **Profil** → Informations du compte + déconnexion
- **Connecté en tant qu'Admin** : barre d'onglets avec 6 sections :
 - **Accueil** → Tableau de bord avec compteurs
 - **RDV** → Gestion des rendez-vous avec filtres et actions
 - **Médecins** → Liste → Détail + disponibilités → Formulaire
 - **Patients** → Liste des patients inscrits
 - **Notifications** → Liste des notifications avec badge
 - **Profil** → Informations du compte + déconnexion

ARCHITECTURE EN 3 COUCHES

Le code mobile est organisé en trois couches distinctes pour séparer les responsabilités :

COMPONENTS

Composants UI réutilisables sans logique métier : Bouton, ChampTexte, CarteMedecin, CarteRendezVous, LigneInfo, Chargement. Ils reçoivent données et callbacks via props.

HOOKS

Logique métier extraite dans des hooks : `useAuth` (connexion, inscription, déconnexion), `useMedecins` (liste médecins), `useRendezVous` (liste RDV, statut), `useNotifications` (liste, marquage lu).

SCREENS

Écrans légers qui assemblent composants et hooks. Ils ne contiennent que le layout et la navigation, pas de logique métier directe.

ÉCRANS DE L'APPLICATION

ÉCRANS PATIENT

ÉCRAN	RÔLE
Dashboard	Statistiques des RDV (total, confirmés, en attente) et aperçu des prochains rendez-vous
ListeMedecins	Liste des médecins avec nom et spécialité, clic pour prendre RDV
PriseRendezVous	Prise de RDV en étapes : choix du médecin → date disponible → créneau libre → motif → confirmation
MesRendezVous	Liste de tous les RDV du patient avec badge de statut coloré
DetailRendezVous	Fiche complète d'un RDV avec bouton d'annulation

ÉCRANS ADMIN

ÉCRAN	RÔLE
AdminDashboard	Compteurs cliquables : RDV, en attente, médecins, patients
GestionRendezVous	Tous les RDV avec filtres par statut et actions (confirmer, refuser, remettre en attente)
GestionMedecins	Liste des médecins avec accès au détail
DetailMedecin	Fiche du médecin + gestion des disponibilités (ajout/suppression)

FormulaireMedecin	Ajout ou modification d'un médecin (nom, spécialité)
GestionPatients	Liste des patients inscrits avec suppression

ÉCRANS COMMUNS

ÉCRAN	RÔLE
SplashScreen	Écran d'accueil avec le logo DOC.AGENDA pendant 2 secondes
Connexion	Formulaire email / mot de passe pour se connecter
Inscription	Formulaire de création de compte patient
Notifications	Liste des notifications avec icône par type, date relative, bouton « tout lire »
Profil	Informations du compte, badge de rôle et bouton de déconnexion

COMMUNICATION AVEC L'API

Le fichier `api.js` centralise tous les appels `fetch` vers le backend. Le token JWT est stocké localement avec `AsyncStorage` et automatiquement attaché en en-tête `Authorization: Bearer <token>` à chaque requête. L'état d'authentification est partagé dans toute l'application grâce à un `AuthContext` (React Context). Un `NotificationContext` séparé gère le compteur de notifications non lues avec un rafraîchissement automatique toutes les 30 secondes.

DESIGN

L'application adopte un thème sombre avec une identité visuelle forte : fond `#1a1a2e`, texte crème `#f0ead6`, accent rouge `#e85d4a`, vert médical `#2d6a6a`. La typographie utilise des graisses élevées (900) et un espacement des lettres prononcé pour un rendu moderne et lisible. Les badges de statut utilisent un code couleur intuitif : jaune pour « en attente », vert pour « confirmé », rouge pour « annulé ».

6. FONCTIONNALITÉS RÉALISÉES

GESTION DES RENDEZ-VOUS

FONCTIONNALITÉ	DESCRIPTION
Prendre un RDV	Sélection guidée : médecin → date disponible → créneau libre → motif → confirmation
Consulter ses RDV	Liste défilante avec badge de statut coloré (en attente / confirmé / annulé)
Annuler un RDV	Annulation douce (changement de statut) avec confirmation
Voir le détail	Fiche complète avec nom du médecin, spécialité, date, heure, motif et statut

GESTION DES MÉDECINS (ADMIN)

FONCTIONNALITÉ	DESCRIPTION
Ajouter un médecin	Formulaire avec nom et spécialité
Modifier un médecin	Même formulaire pré-rempli avec les données existantes
Supprimer un médecin	Confirmation via Alert avant suppression
Gérer les disponibilités	Ajout et suppression de plages horaires par jour de semaine

VALIDATION DES RENDEZ-VOUS (ADMIN)

FONCTIONNALITÉ	DESCRIPTION
Filtrer par statut	Filtres rapides : tous, en attente, confirmés, annulés
Confirmer un RDV	Passage du statut « en attente » à « confirmé »
Refuser un RDV	Passage du statut « en attente » à « annulé »
Remettre en attente	Réinitialisation du statut depuis confirmé ou annulé

NOTIFICATIONS IN-APP

ÉVÉNEMENT	DESTINATAIRE	DESCRIPTION
Création RDV	Patient + Admins	Le patient est informé que son RDV est en attente. Les admins reçoivent une alerte.
Confirmation RDV	Patient	Le patient est informé que son RDV a été confirmé.
Annulation (admin)	Patient	Le patient est informé que son RDV a été annulé.
Annulation (patient)	Admins	Les admins sont informés qu'un patient a annulé son RDV.

AUTHENTIFICATION ET SÉCURITÉ

FONCTIONNALITÉ	DESCRIPTION
Inscription	Création de compte patient avec validation des champs
Connexion	Authentification par email / mot de passe, retourne un token JWT
Gestion des rôles	Deux rôles (patient / admin) avec middleware de contrôle d'accès
Persistance de session	Token JWT stocké dans AsyncStorage, vérifié au lancement
Hashage	bcryptjs avec 10 rounds de salage côté serveur

7. CONCLUSION

Ce projet a permis de mettre en pratique l'ensemble des concepts fondamentaux du développement mobile avec React Native : création de composants réutilisables, navigation conditionnelle entre écrans, gestion d'état avec les hooks et les contextes, et communication avec une API REST.

La réalisation va au-delà d'un simple CRUD en intégrant un système complet de gestion de rendez-vous avec des problématiques concrètes : gestion des disponibilités, vérification des conflits horaires, workflow de validation par statut et notifications en temps réel. L'authentification JWT avec gestion des rôles a permis d'aborder les problématiques de sécurité (hashage des mots de passe, tokens, middleware de protection, contrôle d'accès par rôle).

L'architecture MVC adoptée côté backend et l'architecture en trois couches (composants / hooks / écrans) côté mobile facilitent la maintenance et la réutilisabilité du code. Le projet constitue une base solide qui pourrait être étendue avec des fonctionnalités supplémentaires comme les notifications push, un système de rappels de rendez-vous, ou un module de messagerie entre patients et médecins.